Задача №1

В контексте того, что в задаче обсуждается интернет-магазин, а именно таблица с информацией о клиентах, то можно сделать следующее замечания к проделанной работы предыдущего сотрудника:

1) **Отсутствие пароля.**
   В таблице указано, что клиент придумывает свой личный логин (до 30 символов) для входа в свою учетную запись на площадке, но атрибут с паролем попросту отсутствует.
   Это довольно серьезное нарушение безопасности как для клиента, так и для самой площадки: процесс идентификации не соответствует современным требованиям, так как его могут заполучить злоумышленники теми или иными способами.

2) **Отсутствие контактных данных клиента.**
   Ввиду того, что логин клиент придумывает самостоятельно, то велика вероятность, что это может быть что угодно: его никнейм в Steam, кличка со школьных годов или просто всплеск фантазии.
   Необходима хоть какая-либо информация о клиенте (контактный телефон, e-mail) для оперативной связи службы поддержки, а также для рассылок со скидками и предложениями, уведомлениями о этапах доставки отправления и прочей информации (в контексте интернет-магазина это вообще первостепенная задача).
   P.S. Возможно, клиент заполняет информацию «О себе» в отдельном окне и, соответственно, в отдельную таблицу, которые связаны между собой через **client_id**, такой вариант допускается и выглядит логично. Но ввиду первого пункта (Отсутствие пароля) и того факта, что клиент может не заполнять информацию «О себе» по разным причинам, а в самой задаче обсуждается таблица о зарегистрированных пользователях (скорее всего, информация «О себе» просто не была предусмотрена на этапе реализации, а сама регистрация происходила по упрощенному варианту), использование номера телефона/почты в паре с паролем – наилучшее решение как для площадки, так и для клиентов, так как это крайне распространенная практика (даже рутинная).

3) **Отсутствие ограничений целостности и уникальности данных.**
   В текущей схеме объявлены поля, но не заданы правила их заполнения. Критически важное поле **login**, выступающее идентификатором для входа, не имеет ограничения уникальности (UNIQUE). Это создает прямую угрозу функциональности системы, так как может привести к

созданию нескольких клиентов с одинаковым логином и последующему конфликту при аутентификации. Кроме того, отсутствуют ограничения NOT NULL для полей, которые по смыслу должны быть обязательными (например, **login** и **reg_date**), что может привести к появлению в базе записей с неполными или некорректными данными.

4) **Отсутствие явной связи с таблицей городов.**
   Поле **city_id** подразумевает, что существует отдельный справочник городов. Однако в схеме не задекларирована связь (FOREIGN KEY) между таблицей **clients** и этой гипотетической таблицей. Это нарушает ссылочную целостность данных: в **city_id** может быть сохранен любой произвольный номер, даже если соответствующего города не существует. Для корректного анализа клиентской базы по географическому признаку и обеспечения чистоты данных такая связь является необходимой.

5) **Недостаточные гарантии корректности значений в ключевых полях.**
   Схема допускает ввод любых числовых значений в поле **age**, без проверки на адекватность (например, отрицательный возраст или значение, превышающее разумные пределы). Аналогично, неясен статус поля **client_id**: заявлено как «уникальный id», но не определено в качестве первичного ключа (PRIMARY KEY), что является стандартной и обязательной практикой для однозначной идентификации записи. Без этих фундаментальных ограничений достоверность данных в системе и надежность работы с ними существенно снижаются.

Итоговый код запроса (MySQL) с исправлениями, согласно вышеперечисленным ошибкам, будет выглядеть следующим образом (Рис.1.1). Полученные таблицы теперь более корректны для работы: введены условия обязательного заполнения для имени, фамилии, e-mail (заменят логин в старой таблице), пароля и даты; добавлены условия уникальности для телефона и e-mail (получаем связь 1 клиент – 1 учетная запись); добавлена функция автосоздания уникальных **client_id** через AUTO_INCREMENT, проверка возраста в диапазоне от 14 до 120 лет (убираются мусорные данные), а датой регистрации клиента станет дата проведения операции регистрации по умолчанию. Итоговые таблицы можно рассмотреть на Рис.1.2.

```sql
CREATE TABLE cities (
    city_id INT PRIMARY KEY,
    city_name VARCHAR(255) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE clients (
    client_id INT PRIMARY KEY AUTO_INCREMENT,
    client_name VARCHAR(255) NOT NULL,
    client_surname VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    phone VARCHAR(20) UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    city_id INT NULL,
    age INT CHECK (age >= 14 AND age <= 120),
    reg_date DATE NOT NULL DEFAULT (CURRENT_DATE),
    FOREIGN KEY (city_id) REFERENCES cities(city_id) ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Рис.1.1. Запрос на создание таблиц cities и clients.

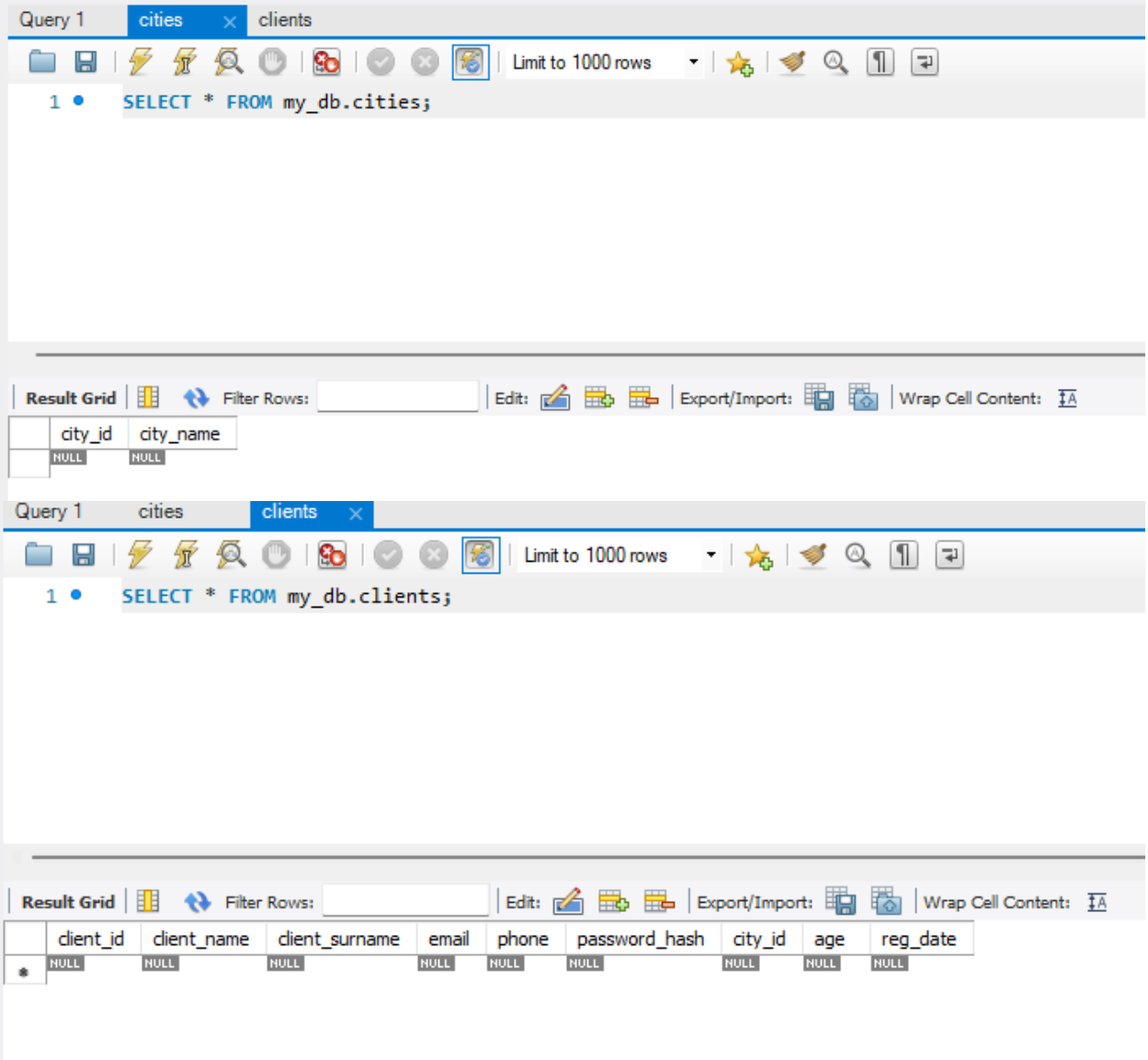

Рис.1.2 – Таблицы cities и clients

Задача №2

Изменение цены, как и изменения в коде проекта, можно фиксировать через систему коммитов, что крайне удобно. Сохранение информации о стоимости какого-либо товара нужно производить на полном цикле его введения, реализации и снятии с продаж (опционально). В данном же случае подобную систему по отслеживанию изменений можно построить следующим образом:

1) Создать в таблице **items** новый атрибут, который будет показывать дату изменения времени (**last_price_update**);
2) Создать новую таблицу истории цен (**price_history**), которая будет связывать с таблицей товаров через **item_id**, изменение цены которого будет записано как событие под **price_id** (это условное обозначение операции, ее порядковый номер).

В таком виде мы имеем две таблицы для разных задач: **items** – для быстрой и оперативной работы (например, изменению цены в карточках товаров руками модератора) и **price_history** – для аналитики и отчетности (например, построить график изменения цены по датам на определенный товары(ы), вычислить суммарный доход за определенный месяц, узнать частоту изменения цен по определенным причинам и так далее).

Ввиду того, что **price_history** – это та таблица, которая должна пополняться и хранить в себе данные не один год, будет разумно не переполнять ее излишними атрибутами, но и указать необходимый минимум параметров, которые покроют большую часть запросов в будущем. К таким атрибутам можно отнести:

- **price_id** – уникальный номер операции изменения;
- **item_id** – уникальный номер товара, над которым были совершенны изменения;
- **old_cast** – старая цена на момент изменения;
- **new_cost** – новая цена в результате изменения;
- **change_date** – дата изменения;
- **changed_by** – кто совершил изменение;
- **reason** – причина изменения.

Для того, чтобы создать эти таблицы, потребуется следующий запрос для MySQL (Рис.2.1):

```sql
CREATE TABLE items (
    item_id INT PRIMARY KEY AUTO_INCREMENT,
    item_name VARCHAR(255) NOT NULL,
    item_cost DECIMAL(10, 2) NOT NULL,
    last_price_update DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE price_history (
    price_id INT PRIMARY KEY AUTO_INCREMENT,
    item_id INT NOT NULL,
    old_cost DECIMAL(10, 2) NOT NULL,
    new_cost DECIMAL(10, 2) NOT NULL,
    change_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    changed_by VARCHAR(100),
    reason VARCHAR(255),
    FOREIGN KEY (item_id) REFERENCES items(item_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

Рис.2.1 – Запрос создания таблиц items и price_history

Чтобы история цен была правильной, а текущая цена корректной, нужно следовать следующему алгоритму по изменению цены в рамках одного запроса (для удобства и согласованности дат, если она ставится по умолчанию (т.е. на данный момент, а не определенная)):

1) Указать **item_id**, для которого изменяется цена;
2) Указать старую цену в **old_cost** в **price_history**;
3) Указать новую цену в **new_cost** в **price_history**;
4) Указать новую цену в **item_cost** в **items**;
5) Указать нужную дату (или текущую через CURRENT_TIMESTAMP);
6) Указать того, кто изменил цену (опционально);
7) Указать причину изменения (опционально)

Пример запроса, производящего изменения цена согласно алгоритму, представлен ниже (Рис 2.2):

```sql
INSERT INTO price_history (item_id, old_cost, new_cost, changed_by, reason, change_date)
SELECT
    3,
    item_cost,
    139990.00,
    'admin',
    'Сезонная скидка',
    CURRENT_TIMESTAMP
FROM items WHERE item_id = 3;

UPDATE items
SET
    item_cost = 139990.00,
    last_price_update = CURRENT_TIMESTAMP
WHERE item_id = 3;
```

Рис. 2.2 – Запрос на изменение цены товара с item_id = 3

Производя тестирование путем добавления товаров и изменения цен получим следующие данные в таблицах (Рис.2.3 и Рис.2.4):

item_id	item_name	item_cost	last_price_update
1	iPhone 17 Pro	129990.00	2026-01-28 10:00:00
2	Samsung Galaxy S26	109990.00	2026-01-25 14:30:00
3	Ноутбук MacBook Air M4	149990.00	2026-01-20 09:15:00
4	Наушники Sony WH-1000XM7	39990.00	2026-01-15 11:45:00
5	Умные часы Apple Watch Series 11	49990.00	2026-01-10 16:20:00
6	Холодильник Bosch KGN45	99990.00	2026-01-05 13:10:00
7	Стиральная машина LG F2V5	64990.00	2025-12-28 10:30:00
8	Посудомоечная машина Siemens iQ700	79990.00	2025-12-20 15:45:00
9	Кофемашина DeLonghi Dinamica Plus	55990.00	2025-12-15 12:00:00
10	Пылесос Dyson V18	69990.00	2025-12-10 09:30:00
11	Куртка зимняя Arc'teryx Beta AR	35990.00	2025-11-25 14:00:00
12	Кроссовки Nike Air Max 2026	15990.00	2025-11-20 11:20:00
13	Джинсы Levi's 501 Premium	9990.00	2025-11-15 10:45:00
14	Пальто шерстяное Max Mara	28990.00	2025-11-10 16:15:00
15	Рюкзак The North Face Borealis	7990.00	2025-11-05 13:30:00
NULL	NULL	NULL	NULL

Рис.2.3 – Заполненная таблица items

price_id	item_id	old_cost	new_cost	change_date	changed_by	reason
1	1	139990.00	129990.00	2025-11-15 10:00:00	marketing	чёрная пятница 2025
2	1	129990.00	134990.00	2025-12-26 09:30:00	manager	послепраздничный рост
3	1	134990.00	129990.00	2026-01-28 10:00:00	auto_system	январские распродажи 2026
4	2	119990.00	109990.00	2025-12-10 11:20:00	launch_promo	предзаказ со скидкой
5	2	109990.00	104990.00	2025-12-28 16:45:00	new_year	новогодняя распродажа
6	2	104990.00	109990.00	2026-01-25 14:30:00	finance	окончание акции
7	3	159990.00	149990.00	2025-09-01 09:00:00	back_to_school	акция "Снова в школу"
8	3	149990.00	144990.00	2025-11-25 13:30:00	black_friday	чёрная пятница
9	3	144990.00	149990.00	2026-01-20 09:15:00	system	возврат к стандартной цене
10	4	44990.00	39990.00	2026-01-15 11:45:00	q4_promo	январская распродажа
11	5	54990.00	49990.00	2026-01-10 16:20:00	tech_sale	технологическая распродажа
12	6	109990.00	99990.00	2026-01-05 13:10:00	jan_sale	январские скидки
13	7	69990.00	64990.00	2025-12-28 10:30:00	singles_day	распродажа перед НГ
14	8	84990.00	79990.00	2025-12-20 15:45:00	winter_sale	зимняя распродажа
15	9	59990.00	55990.00	2025-12-15 12:00:00	dec_sale	декабрьские скидки
16	10	74990.00	69990.00	2025-12-10 09:30:00	pre_xmas	предрождественская распр...
17	11	39990.00	35990.00	2025-11-25 14:00:00	winter_start	начало зимнего сезона
18	12	17990.00	15990.00	2025-11-20 11:20:00	new_model	выход новой модели
19	13	11990.00	9990.00	2025-11-15 10:45:00	black_friday	чёрная пятница
20	14	31990.00	28990.00	2025-11-10 16:15:00	premium_sale	распродажа премиум
21	15	8990.00	7990.00	2025-11-05 13:30:00	accessory_sale	скидка на аксессуары
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Рис.2.4 – Заполненная таблица price_history

Как можно заметить, история всех операций наглядно видна: порядковый номер процесса изменения, старая и новая цена, дата изменения, «кто и почему». С такой таблицей удобно работать и этого набора данных достаточно, чтобы у финансового отдела была возможность строить отчеты за прошедшее время и изменять стратегию продаж на площадке.

Задача №3

В контексте анализа запроса для получения отделов в Сеуле, где сотрудники без менеджера суммарно зарабатывают более 100000, я обнаружил следующие ошибки, которые не позволят добиться желаемого результате при таком требовании, а именно:

1) **Некорректная проверка на NULL.**
   Использование условия MANAGER_ID = NULL является ошибочным, так как NULL представляет собой неопределенное значение. Для корректной проверки необходимо применять оператор IS NULL.

2) **Неправильное расположение агрегатной функции.**
   Условие SUM(SALARY) ≥ 100000 указано в разделе WHERE, что синтаксически неверно. Агрегатные функции могут использоваться только в разделах SELECT или HAVING.

3) **Неполная группировка данных.**
   В секции GROUP BY указан только DEPARTMENT_NAME, тогда как в SELECT также присутствует DEPARTMENT_ID. Все неагрегированные столбцы из SELECT должны быть включены в GROUP BY.

4) **Риск множественного результата в подзапросе.**
   Подзапрос для определения LOCATION_ID в Сеуле может вернуть несколько значений, что приведёт к ошибке выполнения. Более надёжно использовать явное соединение с таблицей LOCATIONS.

5) **Ошибка в названии к обращаемой таблицы**
   В представленном запросе после FROM идет название таблицы EMPLOYEE, хотя ее название, согласно условиям задачи – EMPLOYEES. Возникнет ошибка, так как такой таблицы не существует.

Для корректного решения задачи необходимо переписать запрос с использованием явных JOIN, перенести агрегатное условие в HAVING и указать все неагрегированные поля в GROUP BY.

Исправленный запрос представлен ниже (Рис.3.1):

```sql
SELECT
    D.DEPARTMENT_ID,
    D.DEPARTMENT_NAME,
    SUM(E.SALARY) AS TOTAL_SALARY
FROM DEPARTMENTS D
JOIN EMPLOYEES E ON D.DEPARTMENT_ID = E.DEPARTMENT_ID
JOIN LOCATIONS L ON D.LOCATION_ID = L.LOCATION_ID
WHERE L.CITY = 'SEOUL'
  AND E.MANAGER_ID IS NULL
GROUP BY D.DEPARTMENT_ID, D.DEPARTMENT_NAME
HAVING SUM(E.SALARY) > 100000;
```

Рис.3.1 – Корректный запрос

Задача №4

Будем считать, что карандаш падает случайным образом в некоторой комнате, наполнение которой нам неизвестно, но возможно (столы, стулья, шкафы и так далее). Но мы будем считать, что комната абсолютна пуста, так как задачки подобного рода бывают только такие.

Тогда, моему старту каскадерской карьеры ничего не мешает, кроме того, из чего состоит сама комната, а именно стены. Получается, что я не смогу перепрыгнуть через карандаш, если он будет у одной из стен, в углу, прижмется слишком плотно к стене или же с небольшим отступом – все это помешает мне физически совершить прыжок.